

Key Schedule Guided Persistent Fault Attack

Xue Gong, Fan Zhang[✉], *Member, IEEE*, Xinjie Zhao, Jie Xiao[✉], *Member, IEEE*, and Shize Guo[✉]

Abstract—*Persistent Fault Analysis (PFA)* is a powerful analysis technique proposed in CHES 2018, which utilizes those faults that are injected before execution and persist throughout the encryption. However, when it is applied to the block cipher which has multiple S-boxes, the key cannot be recovered in just one attack. The adversary has to conduct the fault attack several times and inject faults into all the distinct S-boxes. In this paper, we propose *Key Schedule Guided Persistent Fault Attack (KGPF)*, which utilizes the key schedule to guide the fault injection and fault analysis. By analyzing the key schedule, KGPF exploits the relations between the key leakages caused by the same faulty S-box in various rounds. It can reduce the number of attacks and the number of faults required to recover the key. Our major contributions are twofold. Firstly, in the fault injection step, we provide *Key Schedule Guided Persistent Fault Injection (KGPI)* strategies to reduce the number of attacks and the number of faults under the assumption of both ciphertext-only and known-plaintext attacks. Secondly, in the fault analysis step, as our target ciphers are Feistel-based, we propose the *Ineffective Algebraic Persistent Fault Analysis (IAPFA)* to extend the usage of *Algebraic Persistent Fault Analysis (APFA)* in the ineffective persistent fault setting. To demonstrate the effectiveness of our technique, we apply KGPF to four widely used block ciphers with multiple S-boxes, DES, 3DES, LBlock, and Camellia. In our experiment, in the ciphertext-only attack, the key of DES can be recovered with 300 ineffective ciphertexts (corresponding to 827 ciphertexts) and four faulty S-boxes within 12.18min. Under the assumption of known-plaintext, the key of DES is recovered within two faulty S-boxes in 2.34h. For LBlock, the key is recovered with two faulty S-boxes and 100 ineffective ciphertexts (corresponding to 6211 ciphertexts) in 1.16min.

Index Terms—Key schedule, fault attack, PFA, DES, 3DES, Camellia, LBlock.

I. INTRODUCTION

FAULT Attacks (FA) are a class of physical attacks on cryptographic circuits. In FA, the adversary actively

injects faults into the device and recovers the secret key by examining the response of the device to the faults [1], [2], [3]. *Fault Injection (FI)* and fault analysis are the two main steps in FA.

In the fault injection process, faults can be classified as transient [4], permanent [5], or persistent [6] depending on their duration. Trojans, clock glitches [7], laser pulses [8], are examples of methods to introduce transient faults into a circuit. In order to successfully inject a transient fault, the attacker must have exact control over the fault's intensity and duration, which is challenging in real-world applications. Permanent faults are caused by defects in chips that are irrecoverable. Since the permanent defects are typically irreversible, the attackers rarely employ them in practice.

Persistent Fault Attack (PFA) was introduced by Zhang et al. in CHES 2018 for cracking block ciphers [9]. Different from transient faults [7], [8], [10], persistent faults maintain in the device from one encryption to another until the target device reboots. As the fault can be injected before the execution of the encryption algorithm, the adversary does not need to keep synchronization with the algorithm.

The persistent fault attack consists of three steps:

Step 1: Fault Injection. The adversary injects faults before the execution of a block cipher. The faults persist in the device. The affected constant stays faulty unless refreshed. The number of faults injected by one injection can be a single fault or multiple faults.

Step 2: Ciphertexts Collection. The adversary collects ciphertexts generated by the faulty device.

Step 3: Fault Analysis. Through statistical analysis, the adversary acquires the biased distribution of the collected ciphertexts caused by the faults. The adversary then uses the biased distribution to recover the key.

In this paper, *one persistent fault attack* refers to a single loop from Step 1 to Step 3. In practice, fault analysis (Step 3) will be always performed offline and the workload for the online stage will be mainly on Step 1 and Step 2. Therefore, if the adversary needs to repeat from Step 1 to Step 2 for one more time, we consider this case as *two persistent fault attacks*.

A. Related Works

The PFA proposed in [9] is an innovative work. However, there are several questions left to be discussed and some gaps left to be filled in its application. Some follow-up works aim to solve these problems.

Direction 1: The original PFA in [9] is applied to block cipher AES, which is a block cipher of SPN structure. For

Received 2 April 2024; revised 16 July 2024 and 29 September 2024; accepted 22 October 2024. Date of publication 11 November 2024; date of current version 8 January 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106800, in part by the National Natural Science Foundation of China under Grant 62072398, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20220075, and in part by the Natural Science Foundation of Zhejiang Province under Grant LR24F020003. The associate editor coordinating the review of this article and approving it for publication was Dr. Stjepan Picek. (*Corresponding author: Fan Zhang.*)

Xue Gong, Fan Zhang, and Xinjie Zhao are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: xuegong@zju.edu.cn; fanzhang@zju.edu.cn; zhaoxinjieem@163.com).

Jie Xiao is with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310027, China (e-mail: xiaojieqxj@foxmail.com).

Shize Guo is with the College of Control Science and Engineering, Zhejiang University, Hangzhou 310027, China (e-mail: nsfsgsz@126.com).

Digital Object Identifier 10.1109/TIFS.2024.3495234

AES, the fault in the S-box can directly lead to the biased distribution of the ciphertexts. While for Feistel block ciphers, the output of the S-box is XORed by a random input. So, the ciphertexts do not have a biased distribution. Applying PFA on Feistel block ciphers is the first challenge.

Direction 2: In the fault analysis step, the original PFA only utilizes the biased distribution of the ciphertexts, which is the result of the faulty S-box in the last round. As the fault persists in the whole encryption, there are fault leakages in every round. Making full use of the fault leakages is another challenge.

Direction 3: In [9], the fault model is for AES which has a single S-box. The attacks for those block ciphers with multiple S-boxes are not discussed.

For Direction 1, in [11], Caforio and Banik performed the first simple analysis of the Feistel block cipher using PFA. In [12], Bagheri et al. proposed the Statistical Ineffective Persistent Faults Analysis on Feistel Ciphers (SIPFA) which utilizes Statistical Ineffective Fault Analysis (SIFA) [13] in the persistent fault setting. They apply SIPFA on three Feistel schemes, DES [14], 3DES, and Camellia [15].

For Direction 2, in [16], Zhang et al. proposed the Algebraic Persistent Fault Analysis (APFA), which tries to make full use of the free fault leakages in deeper rounds.

But for Direction 3, both the results of [12] and [16] are not so efficient in practice. The three block ciphers analyzed in [12] are all with multiple S-boxes. In one persistent attack, the adversary can only recover 6 or 8 bits of the key. To recover the entire key, the adversary needs to launch several persistent fault attacks. This implies that the adversary needs to go through step 1 and 2, restart the encryption device, and repeat step 1 and step 2 several times. For example, DES has 8 S-boxes, to recover the key of DES, the adversary needs to launch the persistent fault attack 8 times. Respectively, the PFA procedure needs to be repeated 8 times for 3DES, and 4 times for Camellia. In [16], LBlock is a block cipher with 8 S-boxes. They also perform the PFA for 8 times to recover the full key.

B. Motivation and Challenges

In previous works, adversaries targeting block ciphers with multiple S-boxes needed to launch several persistent fault attacks to recover the entire key. Each attack required restarting the device, such as resetting the chip, and injecting a new fault into a different S-box. These actions demanded high privileges and continuous operational permissions, along with advanced techniques for stealth and concealment. This approach contrasts with the original Persistent Fault Attack (PFA), where the adversary only needs to inject a persistent fault once before the victim initiated encryption.

For block ciphers with multiple S-boxes, whether the entire key can be recovered in just one persistent fault attack is the first challenge.

A direct solution to this challenge would be injecting sufficient faults in one attack. However, many block ciphers with multiple S-boxes also have the Feistel-based structure. Injecting sufficient faults in one attack may not lead to a

successful key recovery. Because for Feistel block ciphers, as shown in [12], ineffective ciphertexts are needed for key recovery. If too many faults are injected at the same time, the ineffective ciphertexts rate would be too low, thus collecting enough ineffective ciphertexts would be hard. To guarantee that sufficient ineffective ciphertexts can be collected during the attack, the faults injected in one attack should not be too many.

So, to relax the requirements on operation permissions of the device, our aim is to successfully recover the entire key in one persistent fault attack. That means we inject faults beforehand and wait for the victim to start the encryption just as in the original PFA. We do not restart the device and re-inject new faults during our attack. At the same time, the number of faults should be limited to as fewer as possible to guarantee that sufficient ineffective ciphertexts could be collected. To successfully recover the key under these situations, we need new attacking strategies and new analysis methods.

C. Our Contribution

Our contributions are as follows:

- 1) We propose the *Key Schedule Guided Persistent Fault Attack* (KGPFPA) which consists of *Key Schedule Guided Persistent Fault Injection* (KGPFPI) strategy and *Ineffective Algebraic Persistent Fault Analysis* (IAPFA). KGPFPA can efficiently reduce both the number of required faults and the number of required ineffective ciphertexts in key recovery for those block ciphers with multiple S-boxes.
- 2) In the fault injection step, we propose the technique named as KGPFPI. The adversary can analyze beforehand which S-box to inject the fault and how many faults are needed according to our strategy. For the first time, the relation between key schedule and fault injection is analyzed and exploited.
- 3) In the fault analysis step, we propose *Ineffective Algebraic Persistent Fault Analysis* (IAPFA). As some of the multiple S-boxes ciphers are Feistel based, the output of the S-box is masked by a random bit string. Thus, the adversary has to exploit the input of the S-box instead of the output. To utilize the fault leakages in the input of faulty S-boxes and exploit the relations of fault leakages in different rounds, we combine ineffective persistent fault attacks with algebraic fault analysis and present IAPFA.
- 4) We generalize our attack framework under both ciphertext-only scenario and known-plaintext scenario.
- 5) We conducted attacks on four block ciphers: DES, 3DES, LBlock, and Camellia, all characterized by multiple S-boxes and Feistel structures. In our attack the keys of these ciphers can be recovered within a reasonable timeframe.

D. Comparison With Previous Works

For DES, 3DES, LBlock, and Camellia, we compare our results with two counterparts: APFA [16] and SIPFA [12]. The results are shown in Table I. The term “K/U Faults”

TABLE I
COMPARISON OF DIFFERENT PFA METHODS ON BLOCK CIPHERS WITH MULTIPLE S-BOXES

Algorithm	Attack Scenario	K/U Fault	#Attack	#Fault	#Ineffective Ciphertexts	#Total Ciphertexts	Time	Ref.
DES	Ciphertext Only	Known	1	4	300	827	12.18min	This paper
	Ciphertext Only	Known	8	8	2432	3120	neg.	SIPFA [12]
	Ciphertext Only	Unknown	1	1	2000	2574	$2^{42.97}$ DES calls	SIPFA [12]
3-DES	Ciphertext Only	Known	1	4	300	6172	44.50min	This paper
	Ciphertext Only	Known	8	8	2432	5176	neg.	SIPFA [12]
	Ciphertext Only	Unknown	1	1	2000	4255	$2^{42.97}$ DES calls	SIPFA [12]
Camellia-128	Known Plaintext	Known	1	4	1560	2743	21.66min	This paper
	Ciphertext Only	Known	4	4	6228	7240	neg.	SIPFA [12]
	Ciphertext Only	Unknown	1	1	6500	7483	$2^{49.49}$ Camellia calls	SIPFA [12]
LBlock	Ciphertext Only	Known	1	2	100	6211	1.16min	This paper
	Ciphertext Only	Known	8	8	112	883	-	APFA [16]

indicates whether the attack assumes known or unknown faults, “#Attacks” denotes the number of attacks, and “#Faults” refers to the number of faults required.

Under the assumption of known faults, for DES and 3DES, SIPFA necessitates eight fault attacks, each requiring a chip reset and a new fault injection into a different S-box, totaling eight faults. In contrast, our KGPFA approach, by analyzing the key schedule and employing IAPFA, requires only a single attack with four faults. The required number of ineffective ciphertexts is 300 while in SIPFA that is $304 \times 8 = 2432$. LBlock is analyzed in [16] but not targeted in [12], hence our comparison is with [16], where we require only two faults versus their eight. For Camellia, both SIPFA and our method require four faults, but our approach necessitates only one attack compared to SIPFA’s four, and we require fewer ciphertexts. Regarding unknown faults, SIPFA requires a single fault but entails high time complexity.

II. BACKGROUND

The related background is introduced in this section. We give the general idea of *Persistent Fault Analysis* (PFA), *Algebraic Persistent Fault Analysis* (APFA), and *Statistical Ineffective Persistent Faults Analysis* (SIPFA).

A. Notations

We denote the S-boxes in the algorithm as $S_1, S_2, \dots, S_j$. The input size of each S-box is of w bits. The set of S-boxes is denoted as $\{S_1, S_2, \dots, S_j\}$. Take S_j as an example, the original output of $S_j[l]$ is V . Suppose the fault is injected to $S_j[l]$ (i.e., the fault location in S_j is l). S_j' denotes the faulty S-box. The faulty output will be $S_j'[l] = U$. The fault value is f , i.e., $S_j'[l] = V \oplus f = U$.

In the key schedule, the original key is K . The r -th round key is denoted as SK^r . In the round function, the input of S_j in the r -th round is denoted as X_j^r , each X_j^r is of w bits. If not for a specific round, we omit the superscript r . The total number of rounds is R . Table II shows the notations used in this paper.

In this paper, we follow the definition of *ineffective fault* in SIFA [13], which implies that, for some instances, the fault does not influence the output. In the following sections, the ciphertexts that are not influenced by the fault are called

TABLE II
NOTATIONS USED IN THIS PAPER

Notations	Definitions	Notations	Definitions
P, C	Plaintext/Ciphertext	$S_j[l]$	The l -th element of S_j
$\mathbb{B}$	The block cipher	X_j^r	Input of S_j in the r -th round
n	The block size	f	The fault value
w	Input size of an S-box	V_l	The value of $S[l]$
r	The r -th round	$\mathbb{F}$	The $\mathbb{F}$ function
R	Number of rounds	PL	Permutation
X	The input block	PL^{-1}	Inverse permutation
XL, XR	Left/Right part of input	$Y^r, \tilde{C}$	$PL^{-1}(Y^r), PL^{-1}(C)$
K	The original key	$C_j, \tilde{C}_j$	The j -th element of C and $\tilde{C}$
SK^r	The r -th round key	$\tilde{C}_j^{max}$	$\tilde{C}_j$ with maximum frequency
$S_1, \dots, S_j$	S-boxes	$\tilde{C}_j^{min}$	$\tilde{C}_j$ with minimum frequency

ineffective ciphertexts. The corresponding plaintexts are named as *ineffective plaintexts*.

B. Persistent Fault Analysis

In CHES 2018, Zhang et al. proposed a new fault analysis method based on the persistent fault model for SPN block ciphers [9], called *Persistent Fault Analysis* (PFA). PFA injected the known fault into the S-box, changing a value in the S-box from V to U . Take AES as an example, and we ignore ShiftRows in the last round. When one fault was injected into the S-box of AES (the size of S-box is 256), the probability distribution of the faulty S-box (S') output becomes as follows:

$$Prob(S'[X_j]) = \begin{cases} 0, & \text{if } S'[X_j] = V, \\ \frac{2}{256}, & \text{if } S'[X_j] = U, \\ \frac{1}{256}, & \text{otherwise.} \end{cases} \quad (1)$$

For the last round of AES, the byte of ciphertext C_j can be seen as $C_j = S'[X_j] \oplus SK_j^R$. The collected ciphertexts are under the same key. The distribution of the ciphertext byte is the same as the S-box output. It is shown in Equation (2).

$$Prob(C_j) = \begin{cases} 0, & \text{if } C_j = V \oplus SK_j^R, \\ \frac{2}{256}, & \text{if } C_j = U \oplus SK_j^R, \\ \frac{1}{256}, & \text{otherwise.} \end{cases} \quad (2)$$

By collecting a sufficient number of ciphertexts, the values with the maximal and minimal number of counts in the ciphertext bytes can be found, denoted as C_j^{max} and C_j^{min} , respectively.

There are three methods that can be used to recover the key.

Method 1: Utilizing C_j^{min} . If the original value of faulty S-box V is known and the number of ciphertexts is sufficient, the byte of last round key SK_j^R can be recovered uniquely by $SK_j^R = C_j^{min} \oplus V$.

Method 2: Utilizing C_j^{max} . The occurrence of C_j^{max} is twice as frequent as the other values, and if the fault value U of the faulty S-box is known, then the key can be recovered by $SK_j^R = C_j^{max} \oplus U$.

Method 3: Utilizing the impossible value of the key. Due to $S'[X_j] \neq V$ and $C_j = SK_j^R \oplus S'[X_j]$, the key can be recovered by $SK_j^R \neq C_j \oplus V$.

C. Algebraic Persistent Fault Analysis

In 2022, Zhang et al. combined persistent fault analysis with algebraic analysis and proposed a new approach called *Algebraic Persistent Fault Analysis* (APFA) [16]. APFA mainly utilizes **Method 3** of PFA and extends it to multiple rounds of block ciphers. APFA describes fault information and block cipher in terms of *Satisfiability Problem* (SAT). The core idea of APFA is to translate deep fault information into a generic representation as follows:

$$\begin{aligned} S'[X_j] &\neq V \\ Y &= PL(SB'(X)) \oplus SK_j \\ PL^{-1}(Y) &= SB'(X) \oplus PL^{-1}(SK_j) \end{aligned} \quad (3)$$

So, the round key can be represented as:

$$\tilde{SK}_j \neq \tilde{Y}_j \oplus V, \quad 0 \leq j < \frac{n}{w} \quad (4)$$

In Equation (3), SB and PL are the substitution layer and permutation layer operations in the SPN structure, respectively. V is the impossible value of the faulty S-box. The representation of Equation (4) is the same in different rounds, so the attacker can build the constraint equation of the key for the last multiple rounds.

D. Statistical Ineffective Persistent Faults Analysis

For most SPN block ciphers, the biased distribution in the output of the faulty S-box can lead to a biased distribution in the output of the rounds. For Feistel block ciphers, the input of the round are divided into XL and XR . The round function $\mathbb{F}$ is applied to half of the state $\mathbb{F}(XR, SK^r)$. SK^r is the r -th round key. The result of $\mathbb{F}(XR, SK^r)$ is XORed with XL , which is an unknown random bit string. As a result, the statistical characteristic in the output of the faulty S-box is destroyed.

Ineffective Fault Attack (IFA) [17] takes advantage of those ciphertexts that are not affected by fault. This means although they are encrypted by the faulty cipher, there are some ciphertexts that are always correct. Assuming that the fault location is l , in Feistel block ciphers, the intermediate states X_j are always XORed with the round key SK_j before accessing the S-box. The result of $X_j \oplus SK_j$ does not visit l during the whole

encryption, i.e., $X_j \oplus SK_j \neq l$. Based on this, in 2022, Bagheri et al. proposed an analysis method called Statistical Ineffective Persistent Faults Analysis (SIPFA) [12]. In SIPFA, The keys are recovered by ineffective ciphertexts using $SK_j^R \neq C_j \oplus l$.

III. OVERVIEW

This section provides the fault model and the overview of the proposed *Key Schedule Guided Persistent Fault Attack* (KGPFPA).

A. Attack Presumptions

1) *Fault Model:* During the fault injection phase, the adversary can repeatedly attempt to inject multiple faults until the desired faults are achieved. The injected faults can accumulate and persist, i.e., they are persistent faults. In our attack, we target Feistel-based block ciphers such as DES, 3DES, Camellia, and LBlock. These ciphers contain multiple S-boxes, and we assume that faults are injected into distinct S-boxes. The faults can involve both single and multiple bits faults, and multiple S-boxes may be faulty. Since the adversary exploits fault leakage at the S-box input, the fault locations must be known. The adversary does not utilize the S-box output, so the fault values can remain unknown. The adversary can collect ineffective ciphertexts. For example, in algorithms using Dual Modular Redundancy (DMR) with No Ciphertext Output (NOC), the output results are ineffective ciphertexts. For algorithms without DMR, or with Random Ciphertexts Output (ROC), the adversary can obtain ineffective ciphertexts by encrypting the same set of plaintexts twice, one for the normal encryption and the other for the encryption with faulty S-boxes and comparing the results.

2) *Target Implementations:* Our work focuses on persistent fault attacks targeting the data storage components across various architectures, including SRAM, DRAM, and non-volatile memories. For software implementations, we consider fault injection techniques like row hammer that can induce bit flips in DRAM. For ASIC implementations, we consider physical injection methods like laser fault injection that can alter SRAM contents. While these techniques allow persistent faults to be injected into memories, we acknowledge that our current adversarial model does not encompass other components like arithmetic logic units or cryptographic accelerators.

B. Framework

The framework of our attack is shown in Figure 1. It consists of four steps:

Step 1: Key Schedule Analysis. The adversary analyzes the key schedule of the block cipher and selects a fault injection strategy. This strategy determines the minimum number of faults needed to recover the key and identifies which S-boxes to target for fault injection.

Step 2: Fault Injection. Similar to the fault model in the original PFA, the adversary injects faults into the cryptographic device before encryption. The injected faults persist during the encryption.

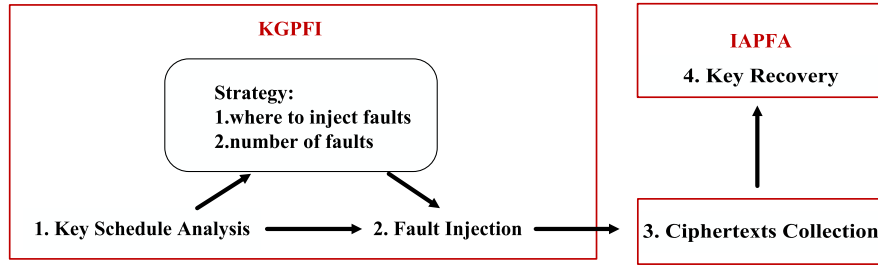


Fig. 1. Framework of KGPF (KGPF=KGPF+IAPF).

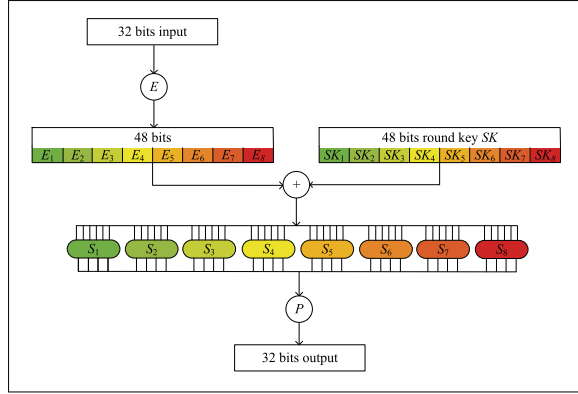


Fig. 2. Round function of DES.

Step 3: Ciphertexts Collection. The adversary collects the ciphertexts. As for Feistel block ciphers, ineffective ciphertexts are needed for key recovery.

Step 4: Fault Analysis/Key Recovery. The adversary performs the offline analysis by doing r rounds of IAPF to recover the key.

C. Core Idea

For block ciphers with multiple S-boxes, the round key is divided into several parts and applied to the input or output of different S-boxes. For example, in Figure 2, the round function of DES, different S-boxes are marked with different colors. The data blocks applying to them are also colored accordingly. The 48 bits round key is divided into 8 parts. Each part is applied to a different S-box.

The key schedule is the algorithm to generate round keys from the original key. The algorithm consists of rotations and permutations. For example, we assume the $\{i_1, i_2, i_3, i_4, i_5, i_6\}$ bits of the original key are applied to the input of S-box S_1 in r -th round. According to the key schedule, some different bits $\{i_7, i_8, i_9, i_{10}, i_{11}, i_{12}\}$ will be applied to the same S-box in the next round. If $\{i_1, i_2, i_3, i_4, i_5, i_6\}$ and $\{i_7, i_8, i_9, i_{10}, i_{11}, i_{12}\}$ are 12 different bits generated from the original key, by utilizing one fault in one S-box and the encryption in two rounds, 12 bits can be recovered.

The significant feature of persistent fault is that the same fault persists in the whole encryption process, thus, utilizing the faulty encryption in different rounds becomes possible. To recover all the bits of the original key, the adversary does

not need to launch several attacks. To recover more original key bits and reduce the number of fault attacks, we propose a novel fault injection strategy, *Key Schedule Guided Persistent Fault Injection Strategy* (KGPF). In KGPF, the adversary can analyze the key schedule before the fault injection and decide the targeted fault position and the number of faults.

By injecting a fault into one S-box, but using the fault leakage in more rounds, we can recover more key bits at a time. In the fault analysis step, to link the fault leakage and the subkey bits in different rounds, *Algebraic Persistent Fault Analysis* (APFA) [16] is an enlightening method. However, in [16], they mainly deal with SPN block ciphers. As some block ciphers of multiple S-boxes are also of Feistel-based structure, APFA can not be used directly on them. So, we extend the algebraic persistent fault analysis to the ineffective fault analysis settings and propose the Ineffective Algebraic Persistent Fault Analysis (IAPFA).

As a whole, KGPF consists of two parts. In the fault injection step, KGPF is proposed to guide the fault injection. In the fault analysis step, IAPFA can be used to exploit information leakage of multiple rounds in Feistel block ciphers.

D. Advantages and Disadvantages

The advantages of KGPF are as follows. First, in KGPF, key recovery can be accomplished in a single attack. The fault injection procedure needs to be launched only once beforehand. This setting implies that the adversary does not require continuous operational authorization to restart the device and re-inject faults. Second, KGPF is effective in both ciphertext-only and known-plaintext scenarios. Third, as demonstrated later, KGPF can bypass some redundancy-based countermeasures. Under certain conditions, redundancy-based countermeasures may even assist the adversary in collecting ciphertexts.

The disadvantage of KGPF is the comparatively low ineffective ciphertexts rate. As for Feistel-based block ciphers, ineffective ciphertexts are required in key recovery. KGPF often needs more than one fault in one attack, thus, the rate of ineffective ciphertexts is lower. So, higher number of encryptions is needed to collect sufficient ciphertexts.

IV. KEY SCHEDULE GUIDED PERSISTENT FAULT INJECTION STRATEGY

In this section, we give the basic principle of the Key Schedule Guided Persistent Fault Injection strategy, which

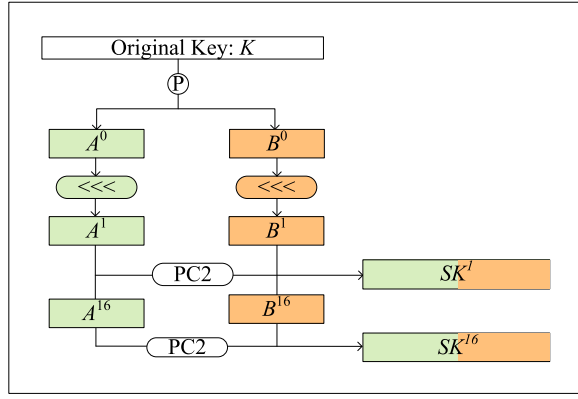


Fig. 3. Key schedule of DES.

TABLE III
ROTATION TABLE (RT)

Iteration Number	1	2	3	4	5	6	7	8
Number of Left Shifts	1	1	2	2	2	2	2	2
Iteration Number	9	10	11	12	13	14	15	16
Number of Left Shifts	1	2	2	2	2	2	2	1

means where to inject the faults and how many faults are needed for those ciphers with multiple S-boxes. The analysis of the key schedule is the foundation of our strategy. We analyzed the key schedules of DES, Camellia, and LBlock. The key schedule of DES is designed in a stream cipher way, whereas the key schedules of Camellia and LBlock are more complicated.

A. Strategy for DES and 3DES

DES is a block cipher with a block size of 64 bits. It consists of an initial permutation, 16 rounds of encryptions, and a final permutation. In the round function, 8 S-boxes are used. The input of each S-box is 6 bits and the output is 4 bits. 3DES is an extended variant of DES. 3DES employs three repetitions of DES. Each repetition has a different key. The key schedule of DES and similarly, 3DES, only use rotations and permutations. Each bit of the round key is directly related to one bit of the original key (denoted as K). The original key consists of 64 bits, of which 56 bits are randomly generated and used directly by the algorithm. One bit in each byte of K may be utilized for error detection in key generation, distribution, and storage. So, the adversary only needs to recover the 56 bits.

The key schedule takes an integer r in the range from 1 to 16 and a 64-bit original key as input and yields as output a 48-bit block SK^r which is a permuted selection of bits from K as in Equation (5).

$$SK^r = \text{KeySchedule}(r, K) \quad (5)$$

SK^r is determined by the bits in 48 distinct positions in K . The calculation of SK^r from the original key K for $r = 1, 2, \dots, 16$ is illustrated in Figure 3.

As shown in Figure 3, the original key is divided into two branches A^0 and B^0 after a permutation. In the following step, each A^{r+1} is generated by left shifts of A^r adhering to the Rotation Table (RT) as in Table III and respectively, each B^{r+1}

TABLE IV
PERMUTED CHOICE 2

14	17	11	24	1	5	3	28	15	6	21	10
23	19	12	4	26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40	51	45	33	48
44	49	39	56	34	53	46	42	50	36	29	32

TABLE V
KEY INDEX OF THE FIRST 6 BITS IN EACH SK^r

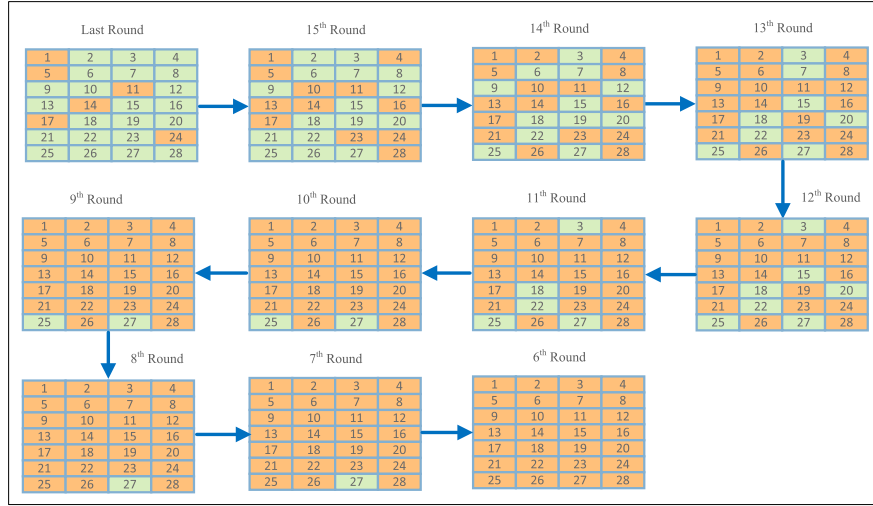
SK^1 :	15	18	12	25	2	6
SK^2 :	16	19	13	26	3	7
SK^3 :	18	21	15	28	5	9
SK^4 :	20	23	17	2	7	11
SK^5 :	22	25	19	4	9	13
SK^6 :	24	27	21	6	11	15
SK^7 :	26	1	23	8	13	17
SK^8 :	28	3	25	10	15	19
SK^9 :	1	4	26	11	16	20
SK^{10} :	3	6	28	13	18	22
SK^{11} :	5	8	2	15	20	24
SK^{12} :	7	10	4	17	22	26
SK^{13} :	9	12	6	19	24	28
SK^{14} :	11	14	8	21	26	2
SK^{15} :	13	16	10	23	28	4
SK^{16} :	14	17	11	24	1	5

is by left shifts of B^r . Permuted Choice 2 is determined by Table IV. The first bit of SK^r is the 14-th bit of $A^r B^r$, the second bit the 17-th, and so on. Notice that the elements of the upper part of the PC2 table are less than or equal to 28, which means all the elements in the upper part of SK^r after PC2 are still from A^r . The elements in the lower part of PC2 are greater than 28, so the lower part of SK^r after PC2 is still from B^r .

Till now, we can see the round keys can also be divided into two independent branches. They are generated by two disjoint sets. When the round keys are applied to the S-box, they are also in a branched way. That is why we call the key schedule of DES as branched key schedule.

Through our analysis, we find out that the round keys of DES and 3DES are generated and used in a branched way. The first branch goes into the first four S-boxes $\{S_1, S_2, S_3, S_4\}$ and the second branch goes into the other four S-boxes $\{S_5, S_6, S_7, S_8\}$. This means that in the persistent fault attack, the faults in the first four S-boxes can only be used to recover the higher 24 bits of round keys which are generated by branch A. So, to recover the full key, at least two faults are needed. The first fault can be in any one of $\{S_1, S_2, S_3, S_4\}$. The second fault can be in any one of $\{S_5, S_6, S_7, S_8\}$.

Now we go one step further to analyze whether two faults are sufficient to recover the entire original key. Let's assume one fault is injected in S_1 and the other in S_5 . We give a simplified description by combining the Rotation Table RT and Permutation Table PC2 together. We list the first 6 bits of each round key with index from A^0 in Table V. For example, the first row in the table is the index of the first 6 bits in SK^1 . The 15-th, 18-th, 12-th, 25-th, 2-nd and 6-th bit of A^0 is used as the first 6 bits in SK^1 . As in Table V, in the last round, the

Fig. 4. Road-map of key leakage from S_1 .

bits 14, 17, 11, 24, 1 and 5 are used as the input of the S-box. With the fault in S_1 and with the last round, these 6 bits can be recovered. If we can go one round deeper, the other 6 bits: 13, 16, 10, 23, 28, and 4 can be recovered.

Figure 4 is the road map of key leakage from S_1 , starting from the last round back to the inner rounds. The orange-colored boxes represent the bits leaked from S_1 in the past rounds. In Figure 4, from the last round till the 6-th round, all the 28 bits will be used as part of the input of S_1 for at least one time. For the adversary, with 11 rounds of IAPFA, 28 bits of A^0 can be recovered by a single fault in S_1 . With another fault injected in one of $\{S_5, S_6, S_7, S_8\}$ at the same time with S_1 , the whole 56 bits of A^0B^0 can be recovered simultaneously. By applying the permutation to A^0B^0 , we can get the original key.

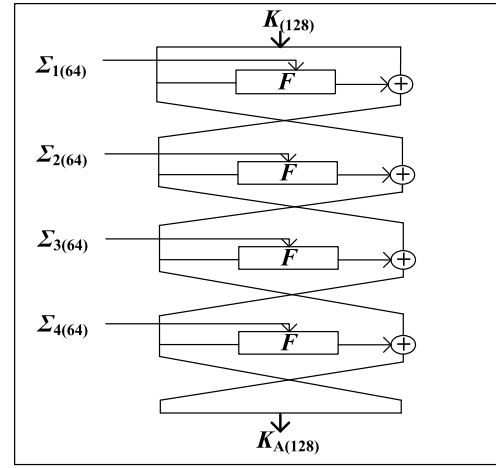
In a nutshell, the *Key Schedule Guided Fault Injection (KGPI)* Strategies for the two-branched key schedules like DES are as follows:

Strategy 1. Two faults injected. The first fault can be in any one of $\{S_1, S_2, S_3, S_4\}$. The second fault can be in any one of $\{S_5, S_6, S_7, S_8\}$.

However, in the algebraic analysis step, the number of algebraic equations and unknown variables for 11 rounds of block ciphers like DES would be a burden for the solvers in practice. We provide some adjusted strategies to reduce the rounds. It is more applicable for those adversaries with limited algebraic solving resources.

Strategy 2. Injecting more faults in one attack. For example, if four faults are injected, two in $\{S_1, S_2, S_3, S_4\}$ and two in $\{S_5, S_6, S_7, S_8\}$, 12 more key bits would be used as the input for the faulty S-box. Thus, more bits can be recovered in each round. The total number of rounds can be reduced.

Strategy 3. Linking the first and last rounds. The adversary who can launch the known-plaintext attacks can use the ineffective plaintexts. However, to recover the keys with 2 faults, the total number of rounds is also 11. The adversary can choose to use the first 6 and last 5 rounds or with the first 5 or last 6 rounds.

Fig. 5. Generation of K_A .

B. Strategy for Camellia

Camellia is a Feistel block cipher proposed in [15]. The block size is 128 bits. It has 18 rounds of Feistel structure for the 128-bit keys, and 24 rounds for 192- and 256-bit keys. It has 4 different S-boxes and each S-box is used twice in the round function. The input of each S-box is 8 bits and the output is also 8 bits. In this paper, our target is the Camellia block cipher of 128-bit keys.

There are two steps of generating the round keys from the original key:

1) *Generation of K_A* : As shown in Figure 5, the 128-bit original key K is used as the input. By taking the 64-bit constants as the keys into the F function, the 128-bit variable K_A is generated by four rounds of the Feistel network.

2) *Generation of the Round Keys*: In Table VI, the 64-bit SubKeys $KW^1, \dots, KW^4, KL^1, \dots, SK^{18}$ are generated from K and K_A . The subkeys are generated by the rotation of K and K_A and taking the left- or right-part.

Due to the big size of the S-boxes, when applying an algebraic attack on Camellia, the number of variables and equations that are generated is high. The theoretical number of

TABLE VI
ROUND KEYS

Function	SubKeys	Value
Prewhtening	KW^1	$(K \lll 0)_L$
Prewhtening	KW^2	$(K \lll 0)_R$
F(Round1)	SK^1	$(K_A \lll 0)_L$
F(Round2)	SK^2	$(K_A \lll 0)_R$
...	...	...
F(Round17)	SK^{17}	$(K \lll 111)_L$
F(Round18)	SK^{18}	$(K \lll 111)_R$
Postwhitening	KW^3	$(K_A \lll 111)_L$
Postwhitening	KW^4	$(K_A \lll 111)_R$

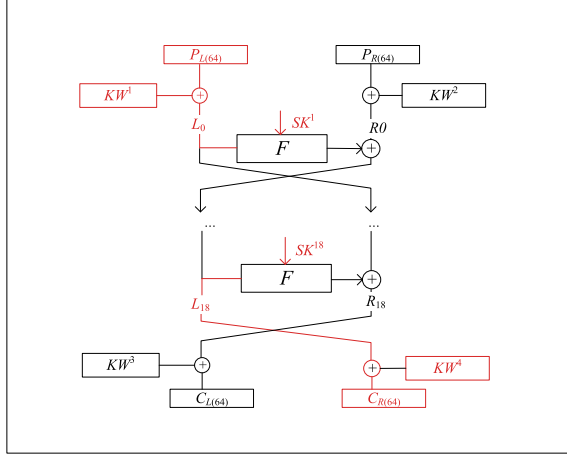


Fig. 6. Encryption procedure of camellia.

ineffective ciphertexts required to recover the full key is about 1561. If we construct the equations for those ciphertexts and use only the last two rounds of Feistel network, the script file size is already more than 1GB. Therefore, it is very difficult to use the fault leakages in deeper rounds. In our attack, we only use the biased access of faulty S-box entries. The results after accessing the S-boxes are not used in our attack. So, using the first round and last round can avoid the representation of S-boxes. Thus, the number of variables and the number of equations are greatly reduced. So, the possible strategy for Camellia is to use the first round and last round under the known-plaintexts attack. Then, the question is, how many bits of the key can be recovered with only the first and last rounds?

Figure 6 is the encryption procedure of Camellia, The data blocks we use are marked in red. before the input of the first round $\mathbb{F}$ function, there is a Prewhtening step. So, the input of the first $\mathbb{F}$ function is $P_{L(64)} \oplus KW^1 \oplus SK^1$.

$P_{L(64)}$ is the left 64 bits of the plaintexts. The input of the last round of the F function is:

$$L_{18} \oplus SK^{18} \quad (6)$$

L_{18} is the left part of the last round input. There is a Postwhitening step after the output of the $\mathbb{F}$ function:

$$L_{18} \oplus KW^4 = C_{R(64)} \quad (7)$$

$C_{R(64)}$ is the right 64 bits of the ciphertexts. By combining Equation (6) and (7) and eliminating L_{18} , the input of the last $\mathbb{F}$ function can be represented as:

$$C_{R(64)} \oplus KW^4 \oplus SK_{18} \quad (8)$$

In the fault analysis step, by using the first and last rounds, we can get the candidate for $(KW^1 \oplus SK^1)$, and $(KW^4 \oplus SK_{18})$. The question following this step is: how many distinct bits can be recovered from $(KW^1 \oplus SK^1)$ and $(KW^4 \oplus SK_{18})$?

In the key schedule, as shown in Table VI, $KW^1 = (K \lll 0)_L$, $SK^1 = (K_A \lll 0)_L$. So we have:

$$KW^1 \oplus SK^1 = ((K \oplus K_A) \lll 0)_L. \quad (9)$$

And $KW^4 = (K_A \lll 111)_R$, $SK^{18} = (K \lll 111)_R$. So we have:

$$KW^4 \oplus SK^{18} = ((K \oplus K_A) \lll 111)_L. \quad (10)$$

Let $K_{new} = K \oplus K_A$. We can recover the K_{new} .

With 4 faulty S-boxes and applying IAPFA on the first round and last round, we can recover $(K_{new} \lll 0)_L$ and $(K_{new} \lll 111)_R$. Totally 111 bits of K_{new} can be recovered. By enumerating the 17 bits left and solving the equations of the generation of K_A , we can get K and K_A . Thus, the original key and the round keys can be recovered.

To summarize, the strategy for Camellia is four faults injected and using the first round and last round under the known-plaintexts attack.

C. Strategy for LBlock

LBlock is a Feistel-based block cipher proposed by Wu et al. in CANS 2011 [18]. The block size of LBlock is 64 bits and the key size is 80 bits. The encryption has 8 S-boxes and 32 rounds. The input of each S-box is 4 bits and the output is also 4 bits.

The 80-bit original key of LBlock is denoted as $K = k_{79}k_{78}k_{77}k_{76} \dots k_1k_0$. The first round key SK^1 is generated by outputting the leftmost 32 bits of the original key. Then for the key of the rest rounds:

For $i = 1, 2, \dots, 31$:

- (1) $K \lll 29$
- (2) $[k_{79}k_{78}k_{77}k_{76}] = S_9[k_{79}k_{78}k_{77}k_{76}] \quad [k_{75}k_{74}k_{73}k_{72}] = S_8[k_{75}k_{74}k_{73}k_{72}]$
- (3) $[k_{50}k_{49}k_{48}k_{47}k_{46}] \oplus [i]_2$

$\lll$ is the left cyclic shift operation. S_8 and S_9 are two 4-bit S-boxes. The key schedule outputs the leftmost 32 bits of K as round key SK^{i+1} .

From the key schedule of LBlock, we can see that the key bits are generated and used as the input of the S-box equally through rotations. Theoretically, one fault in any of the S-box is sufficient for the adversary to recover the whole 80 bits of the original key.

As the input of each S-box is 4 bits, with one faulty S-box, only 4 bits can be recovered in each round. Totally, 20 rounds are needed for the 80 bits original key. Considering the limited scale of algebraic solving resources, adjusted strategies of adding more faults or launching the known-plaintext attack can be considered, just as **Strategy 2** and **Strategy 3** for DES.

TABLE VII
INEFFECTIVE FAULTY CIPHERTEXTS RATE AND THE NUMBER
OF ENCRYPTION CALLS

Block Cipher	No. of Ciphertexts	No. of Faults	Rate of Ineffective Ciphertexts	No. of Encryption Calls
DES	300	2	$(1 - \frac{1}{2^6})^{16 \times 2} = 60.41\%$	$300 \div 60.41\% \simeq 496$
		4	$(1 - \frac{1}{2^6})^{16 \times 4} = 36.49\%$	$300 \div 36.49\% \simeq 822$
3DES	300	2	$(1 - \frac{1}{2^6})^{48 \times 2} = 22.05\%$	$300 \div 22.05\% \simeq 1360$
		4	$(1 - \frac{1}{2^6})^{48 \times 4} = 4.86\%$	$300 \div 4.86\% \simeq 6172$
Camellia	1561	2	$(1 - \frac{1}{2^8})^{18 \times 4} = 75.4\%$	$1561 \div 75.4\% \simeq 2069$
		4	$(1 - \frac{1}{2^8})^{18 \times 8} = 56.9\%$	$1561 \div 56.9\% \simeq 2743$
LBlock	50	2	$(1 - \frac{1}{2^4})^{32 \times 2} = 1.61\%$	$50 \div 1.61\% \simeq 3105$
		4	$(1 - \frac{1}{2^4})^{32 \times 4} = 0.026\%$	$50 \div 0.026\% \simeq 192307$

V. CIPHERTEXTS COLLECTION

For Feistel block ciphers, the adversary needs sufficient ciphertexts to recover the key. In different KGPI strategies, the numbers of faulty S-boxes are different. So, the ineffective ciphertexts rates are different. In this section, we give the detailed calculation of ineffective ciphertexts rate in each strategy.

A. Ineffective Ciphertexts Rate

Suppose the fault is injected into $S_j[l]$. The input of the faulty S-box is denoted as X_j^r . For the ineffective ciphertext, the fault location is not accessed in any round. So we have: $X_j^r \neq l$, $r = 0, 1, 2, \dots, R$. For a cipher with R rounds, if only one element in one S-box is faulty, the rate of ineffective ciphertexts can be estimated based on: $Pr(C) = (1 - \frac{1}{2^w})^R$. If t S-boxes are faulty, each with one faulty element, the ineffective ciphertexts rate would be estimated using the equation: $Pr(C) = (1 - \frac{1}{2^w})^{t \cdot R}$.

For example, the block cipher DES is of 16 rounds and 8 S-boxes. The input of each S-box is of 6 bits. We assume a fault is injected in S_1 . The ineffective ciphertexts rate can be estimated as $(1 - \frac{1}{2^6})^{16} = 77.7\%$. For Camellia, as each S-box is used twice in the round function, assuming a fault in S_1 , the rate will be $(1 - \frac{1}{2^8})^{18 \times 2} = 86.9\%$.

For other ciphers and other KGPI strategies, the rate of ineffective ciphertexts is in Table VII.

B. Estimation of Required Number of Ciphertexts

The number of ciphertexts required to observe h different values is similar to the popularly known coupon collector's problem [19]. As it is analyzed in [9] and [20], the number of ciphertexts required to collect $2^w - 1$ values at the input of an S-box can be estimated by $(2^w - 1) * (\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{2^{w-1}})$.

For DES and 3DES, the input of an S-box is 6 bits, where $w = 6$. The number of ineffective ciphertexts required to eliminate the $2^6 - 1 = 63$ impossible key values can be estimated by: $63 * (\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{63}) \simeq 298$.

This means it needs approximately 300 ineffective ciphertexts to get the right value of the first 6 bits of SK^r for DES and 3DES. One of the advantages of our attack is that the same 300 ciphertexts can be used to recover other bits of K . We just need to collect 300 ciphertexts only once. No matter how many faults are injected, the total number of ineffective

ciphertexts needed to determine the round key is the same. For Camellia, $w = 8$, it needs approximately 1561 ineffective ciphertexts. For LBlock, $w = 4$, it needs approximately 50 ineffective ciphertexts.

For different KGPI strategies with different numbers of faulty S-boxes, the numbers of needed ineffective ciphertexts are the same. The only difference is the ineffective ciphertexts rate. More faulty S-boxes result in a lower rate of ineffective ciphertexts. Thus collecting the same number of ineffective ciphertexts needs more encryption calls. Table VII is the detailed calculation of the ineffective ciphertexts rate and the required encryption calls to collect a certain number of ciphertexts.

C. Collection of Ineffective Ciphertexts

In a ciphertext-only attack, the adversary needs to know which ciphertext is ineffective. The Dual Modular Redundancy (DMR) provides the adversary a bridge towards this process.

Dual Modular Redundancy (DMR) is a mechanism designed to protect the device against fault attack. For SPN block ciphers, it increases the difficulty of persistent fault analysis. In contrast to this, the DMR can be utilized by the adversary to collect ineffective ciphertexts in our attack.

In the redundant encryption based DMR (REDMR), the plaintext is encrypted twice. Provided that the resultant ciphertexts of two modules are the same ($C_1 = C_2$), the security check for REDMR is passed. Thus, the ciphertext is considered true and can be displayed.

In the inversive decryption based DMR (IDDMR). The plaintext is encrypted by Module 1 and then decrypted by Module 2. If the decrypted plaintext from Module 2 is the same as the original plaintext ($P_1 = P_2$), the ciphertext of Module 1 is considered true and can be displayed.

If the ciphertexts failed to pass the security check, three methods can be used:

- (1) No ciphertext output (NCO). If $P_1 \neq P_2$ is detected, the device will not display the incorrect ciphertext C .
- (2) Zero value output (ZVO). If $P_1 \neq P_2$ is detected, the device will display the output that is a ciphertext $C = 0$.
- (3) Random ciphertext output (RCO). If $P_1 \neq P_2$, a ciphertext with random value will be the output.

In our attack, for the devices with REDMR or IDDMR, the adversary injects persistent faults in just one of the modules. In the NCO situation, as the device will not display the incorrect ciphertexts, all the ciphertexts displayed are ineffective ciphertexts. In the ZVO situation, nonzero ciphertexts can be collected by the adversary as ineffective ciphertexts.

For the RCO situation or block ciphers without DMR, the ineffective ciphertexts can be collected by the following steps:

- (1) The adversary encrypts some plaintexts and collects a set of ciphertexts $\{C_1, C_2 \dots C_n\}$.
- (2) The adversary injects faults into the cryptographic device.
- (3) Use the faulty device to encrypt the same set of plaintexts. Collect another set of ciphertexts $\{C'_1, C'_2 \dots C'_n\}$.

By comparing the ciphertexts in $\{C_1, C_2 \dots C_n\}$ and $\{C'_1, C'_2 \dots C'_n\}$ one by one, the adversary can get the ineffective

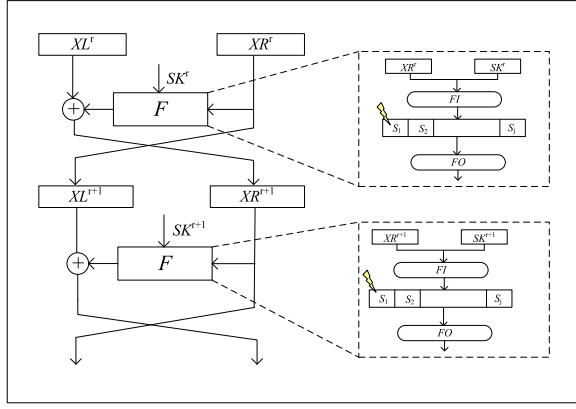


Fig. 7. An illustration of two generalized rounds of Feistel block cipher.

ciphertexts. For example, if $C'_1 = C_1$, C'_1 is an ineffective ciphertext.

VI. INEFFECTIVE ALGEBRAIC PERSISTENT FAULT ANALYSIS

As an algebraic analysis method, IAPFA consists of two steps: building the equations and solving the equations. Since the SAT solvers like Minisat [21], CryptoMiniSat [22], etc. can be used to solve the equations automatically, we do not go into details on the equation-solving step. We use CryptoMiniSat to solve the equations in IAPFA, and we will show our experimental result in Section VII.

In this section, we focus on the first step: building equations for IAPFA. The equation system of IAPFA has three parts: the representation of ineffective fault leakage in deeper rounds, the representation of Feistel block cipher, and the representation of key schedule.

A. Representation of Ineffective Fault Leakage

Suppose the block cipher $\mathbb{B}$ has R rounds. Figure 7 shows the general structure of the Feistel block ciphers. XL^r and XR^r are the left and right parts of the input block in the r -th round. The r -th round encryption can be represented by Equation (11) and (12):

$$XR^{r+1} = \mathbb{F}(SK^r, XR^r) \oplus XL^r \quad (11)$$

$$XL^{r+1} = XR^r \quad (12)$$

The function $\mathbb{F}$ consists of permutations and substitutions. We denote FI as the function before the input of S-boxes and FO as the function after the output of S-boxes. As FI and FO are linear, we can just apply their inverse function to the result of our analysis. For simplicity, we omit it in the following analysis.

Suppose the fault is injected to $S_1[l]$ (i.e., the fault location in S_1 is l). The first block in XR^r is denoted as XR_1^r . XR_1^r is used as the input of S_1 . And respectively, the first block in SK^r is SK_1^r . For the ineffective ciphertext, the faulty S-box entry is not accessed in any round. So we have Equation (13).

$$XR_1^r \oplus SK_1^r \neq l \quad (13)$$

Then we have the ineffective fault representation for S_1 as in Equation (14). For other S-boxes, the equations are the same.

$$SK_1^r \neq l \oplus XR_1^r \quad (14)$$

When applied to different rounds, the only difference is the round index r . This allows us to add the fault ineffective information into the algebraic equation system.

When using SAT solvers like Minisat [21], CryptoMiniSat [22], etc. all the representations should be in the form of *Conjunctive Normal Form* (CNF) and XOR operations. Equation (15) is an example of the conjunctive normal form of two variables. The two variables a and b are with the value true or false.

$$(a \vee \neg b) \wedge \neg a = \text{true} \quad (15)$$

To solve this Boolean equation, we have $a = \text{false}$ and $b = \text{false}$. The conjunctive normal form is composed of several disjunctive clauses, in Equation (15), $(a \vee \neg b)$ is a disjunctive clause.

To use SAT solvers in our algebraic attack, we need to convert the equations into a set of CNF or XOR clauses. The representation of the ineffective fault leakage is to transform Equation (14) to CNF or XOR clauses. In the following description, “ $\cdot$ ” means the bitwise AND. “ $\cdot$ ” can be omitted sometimes for simplicity. For example, $x_i \cdot x_j$ can be simplified as $x_i x_j$. $x_i + x_j$ is the bitwise addition.

We denote the bitwise representation of SK_j^r as follows, where w is the input size of the S-box.

$$SK_j^r = k_0 \| k_1 \| \cdots \| k_{w-1} \quad (16)$$

We denote the right side of Equation (14) as Y_j which means $l \oplus XR_j^r = Y_j$. The representation of Y_j is:

$$Y_j = y_0 \| y_1 \| \cdots \| y_{w-1} \quad (17)$$

Then, $SK_j^r \neq l \oplus XR_j^r$ is represented as in Equation (18).

$$k_0 \| k_1 \| \cdots \| k_{w-1} \neq y_0 \| y_1 \| \cdots \| y_{w-1}. \quad (18)$$

Equation (18) can be transformed to CNF or XOR clauses as follows. $I_0, I_1, \dots, I_{w-1}$ are the intermediate variables introduced in this transform process. Each I_x can be regarded as one bit of $SK_j^r \oplus Y_j$. Since $SK_j^r \neq Y_j$, there should be at least one element that is equal to “1” among $I_0, I_1, \dots, I_{w-1}$. In another words, $I_0, I_1, \dots, I_{w-1}$ cannot be all zeros. So, we have $I_0 \vee I_1 \vee \cdots \vee I_{w-1} = 1$ in Equation (19).

$$\begin{aligned} k_0 + y_0 + I_0 &= 0 \\ k_1 + y_1 + I_1 &= 0 \\ &\vdots \\ k_{w-1} + y_{w-1} + I_{w-1} &= 0 \\ I_0 \vee I_1 \vee \cdots \vee I_{w-1} &= 1 \end{aligned} \quad (19)$$

B. Representation of Feistel Block Cipher and Key Schedule

The basic operations in the block cipher and the key schedule are the same: Bitwise Addition, Substitution, and Permutation.

1) *Representation of Bitwise Addition*: Bitwise Addition is used in key addition and the operation of $XR \oplus XL$ etc. Suppose the input blocks are X and Y . The output is Z . x_i , y_i and z_i are one bit of X , Y , and Z , respectively. The XOR clauses as in Equation (20) can be added to the solver to represent the bitwise add.

$$x_i + y_i + z_i = 0 \quad 0 \leq i < n \quad (20)$$

2) *Representation of Substitution*: The substitution layer is the non-linear function which can be used both in the round function $\mathbb{F}$ and the key schedule. The S-boxes are used as the truth tables to map the value of X_i to Y_i . The core idea is to represent each bit of Y_i with the bits from X_i . It cannot be added to the solver directly. The S-box can be transformed into CNF in two steps according to [23]. Firstly, the S-box is represented with *Algebraic Normal Form* (ANF). Secondly, the ANF is transformed into CNF.

a) *Algebraic Normal Form*: X_i and Y_i are the input and output of the S-box. The bitwise representation are $X_i = \{x_0 || x_1 || \dots || x_{w-1}\}$ and $Y_i = \{y_0 || y_1 || \dots || y_{w-1}\}$. Every bit of Y_i can be represented by the bits in X_i in ANF as shown in Equation (21).

$$y_i = \sum_{p=0}^{2^w-1} a_p x_0^{t_0^p} x_1^{t_1^p} \dots x_{w-1}^{t_{w-1}^p} \quad (21)$$

where $p = \{t_0^p || t_1^p || \dots || t_{w-1}^p\}$ is the binary expansion of p . a_p is the coefficient where $a_p \in \{0, 1\}$. Also $t_i^p \in \{0, 1\}$, where $x_i^{t_i^p}$ ($t_i^p = 1$) means the bit x_i appears in the monomial $a_p x_0^{t_0^p} x_1^{t_1^p} \dots x_{w-1}^{t_{w-1}^p}$, and $x_i^{t_i^p}$ ($t_i^p = 0$) means x_i does not appear.

b) *Conjunctive Normal Form*: The bitwise AND operations are not supported by SAT solvers. The method in [22] is to use intermediate variables to represent the monomials such as $x_0 x_2 x_3$ that have AND operations. The intermediate variable is denoted as I_p .

$$I_p = x_0^{t_0^p} x_1^{t_1^p} \dots x_{w-1}^{t_{w-1}^p}, \quad (p = 0, \dots, 2^w - 1) \quad (22)$$

To add Equation (22) into the algebraic system, the equation below is used:

$$(x_0 \vee \neg I_p) \wedge (x_1 \vee \neg I_p) \wedge \dots \wedge (x_{w-1} \vee \neg I_p) \wedge (\neg x_0 \vee \neg x_1 \vee \dots \vee \neg x_{w-1} \vee I_p) = 1 \quad (23)$$

By adding the constraint of Equation (23) to the solver, for every different value of x_i , the value of I_p will be equal to the term $x_0^{t_0^p} x_1^{t_1^p} \dots x_{w-1}^{t_{w-1}^p}$. For the S-box with w bits input, by using $2^w - 1$ intermediate variables and adding $2^w - 1$ clauses into the solver, the ANF of the S-box is transformed to CNF.

3) *Representation of Permutation*: In the Feistel block cipher, the Rotations, Extensions, and Permutations can be viewed as table replacements. The permutation table T_p is used to represent the relationship between input and output:

$$x_i + y_{T_p[i]} = 0 \quad 0 \leq i < n \quad (24)$$

TABLE VIII
SIMULATION RESULT ON DES

Simulation	Faulty S-boxes	Assumption	#Ineffective Ciphertexts	#Total Ciphertexts	#Plaintexts	Rounds	Time
1	S_1, S_5	Ciphertext-only	300	496	-	11	>48h
2	S_1, S_2, S_5, S_6	Ciphertext-only	300	827	-	4	12.18min
3	S_1, S_5	Known-plaintext	300	496	300	6	2.34h
4	S_1, S_2, S_5, S_6	Known-plaintext	300	827	300	4	2.6s

VII. SIMULATION RESULTS

In this section, we introduce the result of our attack on DES, 3DES, LBlock, and Camellia. The feasibility of achieving single faults on different S-boxes is proven by our Rowhammer fault injection experiment in Section VIII. To compare different strategies, we give the simulation results on fault analysis for different strategies. In this section, we focus on the simulation of key recovery by IAPFA. Firstly, we give a detailed description of our simulation setup. Then we launch our attack on DES, 3DES, and Camellia.

A. Simulation Setup

Our key recovery experiments are implemented on a PC with 16GB memory and an Intel i7-12700H CPU at 2.3GHz. The operating system is a 64-bit Windows 11. The experiments consist of the following steps: (1) We choose the fault injection strategy based on the KGPFA and simulate the fault injection via software. (2) We simulate a victim which implements the block cipher with REDMR. (3) We collect the ciphertexts. (4) We conduct the offline analysis by IAPFA. In IAPFA, we first construct the system of algebraic equations for the key schedule and certain rounds of encryption or decryption. Then we add the ineffective fault information to the system. Finally, we associate the actual ineffective ciphertexts with the equation system and solve the algebraic system using CryptoMiniSAT.

B. Attack on DES

We carry out three attack experiments to compare the three strategies in Section IV-A.

Experiment 1. We exploit strategy 1 and inject two faults, one in S_1 and the other in S_5 . We use 300 ineffective ciphertexts to build the equations of the key schedule, the 11 rounds of decryption, and the fault ineffective information. However, the SAT solver could not give the right solution in a tolerant time (within 48 hours).

Experiment 2. As in strategy 2, faults are injected into four S-boxes: S_1, S_2, S_5 , and S_6 . We use 300 ineffective ciphertexts and the last 4 rounds and get the correct key in 12.18min.

Experiment 3. As in strategy 3, we add the 300 corresponding plaintexts to the system. We use the first 3 and the last 3 rounds to build the equations. The original key is recovered in 8439 seconds which is 2.3 hours.

Experiment 4. We add the 300 corresponding plaintexts to Experiment 2. We use the first 2 and the last 2 rounds to build the equations. The original key is recovered in 2.6 seconds.

The result of the attack on DES is in Table VIII.

TABLE IX
SIMULATION RESULT ON 3DES

Simulation	Faulty S-boxes	Assumption	#Ineffective Ciphertexts	#Total Ciphertexts	#Plaintexts	Rounds	Time
1	S_1, S_2, S_5, S_6	Ciphertext-only	300	6172	-	4	44.50min

TABLE X
SIMULATION RESULT ON CAMELLIA

Simulation	Faulty S-boxes	Assumption	#Ineffective Ciphertexts	#Total Ciphertexts	#Plaintexts	Rounds	Time
1	S_1, S_2, S_3, S_4	Known-plaintexts	1560	2743	1560	2	21.66min

C. Attack on 3DES

3DES, also known as Triple DES, uses three different 56-bit keys to encrypt data three times. Let $\mathbb{E}$ and $\mathbb{D}$ represent the encryption and decryption process of the DES algorithm, $\mathbf{K}$ represents the key used by the DES algorithm, $\mathbf{P}$ represents plaintext, and $\mathbf{C}$ represents ciphertext. 3DES encryption process is $\mathbf{C} = \mathbb{E}(\mathbf{K}_3, \mathbb{D}(\mathbf{K}_2, \mathbb{E}(\mathbf{K}_1, \mathbf{P})))$. 3DES decryption process is $\mathbf{P} = \mathbb{D}(\mathbf{K}_1, \mathbb{E}(\mathbf{K}_2, \mathbb{D}(\mathbf{K}_3, \mathbf{C})))$.

Its original key is $56 \times 3 = 168$ bits. The strategy for 3DES is similar to DES. Under the ciphertext-only scenario, we first recover the key of the outer DES, which is $\mathbf{K}_3$ with the strategy in DES. And then by $\mathbb{D}(\mathbf{K}_3, \mathbf{C})$, we can get the output of the inner DES, which is $\mathbb{D}(\mathbf{K}_2, \mathbb{E}(\mathbf{K}_1, \mathbf{P}))$. Then we recover $\mathbf{K}_2$. And finally, we can get $\mathbf{K}_1$. As the 3 times of encryption in 3DES use the same S-boxes. The whole recovery uses the same set of ciphertexts. The result is in Table IX.

Notice that the key of the inner DES is different from that of the outer DES. The strategy of linking the first round and last round is not suitable in this situation.

D. Attack on Camellia

As each S-box in Camellia is of 8 bits, it is difficult for the algebraic attack to go into deeper rounds. In the simulation of the attack, we use the strategy of linking the first and last rounds. We only build equations on the first and last rounds. The round key is linked by the equations of the key schedule. As shown in Table X, 1561 ineffective ciphertexts and plaintexts are used. We enumerate the last 17 bits of $\mathbf{K}_{new}$ and for each enumeration, the SAT solver can provide a possible key candidate. By encrypting a plaintext with the key candidate and comparing with the ineffective ciphertext, we can find the right key.

E. Attack on LBlock

For LBlock, we launched two experiments based on the strategies in Section IV-C. The results are in Table XI

Experiment 1. The faulty S-box is S_1 . We use 100 ineffective ciphertexts and 8 rounds of decryption. However, the solution of the SAT solver is not right. The reason for that is the constraint of the equation system is not sufficient for the solver to get the only one right solution. So, it just provides a randomly chosen solution that satisfies all the constraints.

Experiment 2. The faulty S-boxes are S_1 and S_2 . We use 100 ineffective ciphertexts and the last 8 rounds and get the correct key in 70 seconds.

TABLE XI
SIMULATION RESULT ON LBLOCK

Simulation	Faulty S-boxes	Assumption	#Ineffective Ciphertexts	#Total Ciphertexts	#Plaintexts	Rounds	Time
1	S_1	Ciphertext-only	100	793	-	8	Failed
2	S_1, S_2	Ciphertext-only	100	6211	-	8	1.16min
3	S_1, S_2, S_3, S_4	Known Plaintext	57	384614	1	5	2s

Experiment 3. The faulty S-boxes are S_1, S_2, S_3 , and S_4 . The number of ineffective ciphertexts we collected under this strategy is only 57. We use the last 5 rounds and get a key with 58 bits correct. The result shows that the number of ciphertexts is not sufficient for the solver to get all the key bits. 22 bits are left for exhaustive search.

VIII. EXPERIMENT

In this section, we present the experimental results of fault injection using the Rowhammer technique. Rowhammer is a hardware-based security exploit that induces bit alterations in Dynamic Random Access Memory (DRAM) by repetitively accessing memory rows adjacent to a targeted victim row.

As a proof of concept, our experimental result demonstrates that we are able to fault several S-boxes such that every S-box has a single fault. The approach enabling this is to employ the idea of Expressive Rowhammer Attack primitive, as detailed by Lou et al. in [24]. It enables the adversary to persist the bit flips and store the error.

A. Setup

Our experiment is conducted on Ubuntu 12.04.3 LTS environment. The CPU is Intel Core i5 3360M. The attack is on a Samsung DDR3 SDRAM with part number M471B5273DH0-CH9 and 4GB capacity.

The process A is the malicious adversary and the process V is the victim. The victim process performs DES encryption. With the idea of PFA, the adversary A can prepare an attack environment before V's encryption. To efficiently inject faults into the S-boxes using Rowhammer, the adversary employs a tool called DRAMDig [25] to reverse-engineer the physical-to-DRAM address mapping.

B. Attack Procedure

The attack procedure consists of four distinct steps: profiling, allocation, positioning, and hammering.

1) *Profiling*: Rowhammer profiling is conducted before the hammering process to identify vulnerable DRAM locations. Approximately 400,000 flippable points are collected during this phase.

2) *Allocation*: After profiling, the adversary aims to precisely allocate the targeted executable file into victim rows for subsequent hammering. Using the 400,000 flippable points identified during profiling, the adversary pinpoints these vulnerable victim rows that are not allocated and locates the targeted S-boxes at these positions.

3) *Positioning*: Before hammering, the adversary attempts to fill aggressor rows next to the selected victim rows. The adversary continuously spray blank pages into memory. The adversary translates the allocated free memory from virtual addresses to physical addresses and then to DRAM internal

TABLE XII
RESULT OF HAMMERING

No.	Fault Location	Original Output	Faulty Output
1	$S_5[40]$	0xf (1111)	0xe (1110)
2	$S_4[9]$	0x2 (0010)	0x0 (0000)
3	$S_6[37]$	0x8 (1000)	0xc (1100)
4	$S_1[27]$	0xb (1011)	0xf (1111)

locations. The adversary must verify whether the DRAM internal location of each allocated blank page is adjacent to the chosen victim rows. If it is, the blank page is retained; otherwise, it is discarded. The virtual addresses of blank pages in these adjacent rows are filled with vulnerable data patterns.

4) *Hammering*: In this step, the adversary can alternately access different aggressor rows under different hammer methods and patterns with high frequency.

C. Result of Hammering

We monitored the sequence of faults occurring in the S-boxes of DES. The detailed results are shown in Table XII. In the table, fault location indicates that the fault is injected into $S_j[l]$, i.e., the l -th element in S_j . Four faults accumulated within two hours. The faulty S-boxes are: S_5, S_4, S_6, S_1 . As S_1 and S_4 are in the set $\{S_1, S_2, S_3, S_4\}$, and the other two faulty S-boxes are in the set $\{S_5, S_6, S_7, S_8\}$, the result meet the requirement of Strategy 2 of DES.

IX. DISCUSSIONS

In this section, we address several issues related to our algorithm. First, we explore the application of KGPF in the presence of masked countermeasures. Next, we analyze methods to manage the unavoidable multiple faults in a single S-box scenario. Finally, we discuss potential countermeasures for KGPF.

A. KGPF in the Presence of Masked Countermeasures

In this subsection, we discuss the extension of KGPF in the context of masked countermeasures. We use a masked implementation of DES, as presented in [26], as a case study.

As shown in [27], PFA can break some masking countermeasures. The faults in our paper are also persistent, but we use the fact that the faulty S-box entry is not accessed in those ineffective ciphertexts. Therefore, the effectiveness of our attack on masking countermeasures depends on whether the biased access to the faulty S-box entry is mitigated by masking. Persistent fault attacks require the S-box to be implemented using a lookup table, and thus, we only consider this type of masked implementation.

The attack assumptions are consistent with those stated in Section III-A. We assume that faults are injected into the lookup table of the S-box. The number of faults depends on the chosen strategy. For instance, in DES, four faults are injected into four distinct S-boxes.

In [26], they proposed the transformed masking method to protect DES and AES. For DES, a 64-bit message M will

be XOR-ed with a randomly chosen 64-bit mask X at the beginning of DES encryption. Then we start with the value $M \oplus X$. Just before the S-box, an intermediary value is masked with $X_2 = E(X_{1R})$ where:

- X_1 represents the 64-bit value $IP(X)$;
- IP represents the initial permutation;
- X_{1L} (respectively X_{1R}) represents the 32-bit low-weight (respectively high-weight) part of the 64-bit mask X ;
- X_2 represents the 48-bit value $E(X_{1R})$;
- E represents the expansive permutation of a DES round.
- A modified S-box is used, denoted SM-box:

$$SM\text{-}box(A) = S\text{-}box(A \oplus X_2) \oplus P^{-1}(X_{1L} \oplus X_{1R}). \quad (25)$$

In this implementation, SM-box consists of a look-up table, the S-box, which is the same as the original DES S-box. Our faults are injected into the S-box. Now we use the first round to show the fact: the input of the S-box is only related to the keys and the plaintext. It is not affected by the mask.

After the initial permutation, we get $IP(M \oplus X)$. We take the right part as the input of the function E , and get $E(IP(M \oplus X)_R)$. The input of SM-box is

$$E(IP(M \oplus X)_R) \oplus K_1.$$

With Equation (25), we have:

$$\begin{aligned} SM\text{-}Box(E(IP(M \oplus X)_R) \oplus K_1) \\ = S\text{-}Box(E(IP(M \oplus X)_R) \oplus K_1 \oplus X_2) \\ \oplus P^{-1}(X_{1L} \oplus X_{1R}) = S\text{-}Box(E(IP(M)_R) \\ \oplus E(X_{1R}) \oplus K_1 \oplus X_2) \oplus P^{-1}(X_{1L} \oplus X_{1R}). \end{aligned} \quad (26)$$

As $E(X_{1R}) = X_2$, the input of S-box will be $E(IP(M)_R) \oplus K_1$. This is only related to the plaintext and the round key. Similarly, in other rounds, the inputs of S-box are not affected by the mask. So, we can still use the biased input of the S-box to recover the key.

B. Protection Against KGPF

During our research, we analyzed several protection measures. As demonstrated in Section V-C, Dual Modular Redundancy (DMR) is designed to protect devices against fault attacks in SPN block ciphers, but it is ineffective against our attack. Masking is intended to protect against side-channel attacks but can be compromised by PFA and our attack. Performing a health check on the S-box before encryption is an effective method against PFA and our attack. However, current health checks often rely on general hash methods such as SHA, which are time-consuming and inefficient for block cipher systems. Therefore, regarding countermeasures, neither DMR nor masking is effective against our attacks. Integrating a built-in health test of the S-box to verify the algorithm's functionality before block encryptions could serve as a potential defense against our attack.

C. Multiple Faults in a Single S-Box

Due to the limitation of fault injection methods, it is likely that multiple elements within a single S-box will be affected. Even if multiple elements within a single S-box are

unavoidably faulted in some fault injection experiments, this situation can still be exploited by KGPFA. The only impact on our attack is a reduced rate of ineffective ciphertexts. Consequently, during the ciphertext collection phase, the adversary must make more encryption calls to gather a sufficient number of ineffective ciphertexts. As a case study, we assume faults are injected into $\{S_2, S_3, S_7, S_8\}$ of DES. Unfortunately, two elements in S_3 are faulted, resulting in a total of five faults across four S-boxes. This will reduce the rate of ineffective ciphertexts to $(1 - \frac{1}{2^6})^{16 \times 3} \times (1 - \frac{2}{2^6})^{16} = 28.25\%$. To collect 300 ineffective ciphertexts, which is the number required for KGPFA, the adversary needs approximately $300 \div 28.25\% \approx 1061$ encryption calls. While in our Strategy 2, the number is 822.

In IAPFA, when building equations, the additional fault in S_3 contributes an extra equation of ineffective fault leakage for the equation system. Suppose the faults are injected to $S_3[0]$ and $S_3[1]$. The third block in XR^r is denoted as XR_3^r . XR_3^r is used as the input of S_3 . And respectively, the third block in SK^r is SK_3^r . For the ineffective ciphertext, both of the faulty S-box entry (*i.e.*, 0 and 1) are not accessed in any round. So we have $XR_3^r \oplus SK_3^r \neq 0$ and $XR_3^r \oplus SK_3^r \neq 1$.

X. CONCLUSION

While the feasibility of persistent fault analysis for block ciphers with a single S-box has been illustrated in the literature, when it comes to block ciphers with multiple S-boxes, it needs multiple attacks to recover the key. In this paper, we present *Key Schedule Guided Persistent Fault Attack* (KGPFA) which makes use of the key schedule to guide the fault injection and analysis. By analyzing the key schedule, KGPFA utilizes the connections between the key leakages brought on by the same faulty S-box in different rounds. The key can be recovered by only one attack for DES, 3DES, LBlock, and Camellia.

REFERENCES

- [1] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of checking cryptographic protocols for faults," in *Proc. Int. Conf. theory Appl. Cryptograph. Techn.* Cham, Switzerland: Springer, 1997, pp. 37–51.
- [2] P. Dusart, G. Letourneux, and O. Vivolo, "Differential fault analysis on AES," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.* Cham, Switzerland: Springer, Oct. 2003, pp. 293–306.
- [3] M. Joye and M. Tunstall, *Fault Analysis in Cryptography*, vol. 147. Cham, Switzerland: Springer, 2012.
- [4] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, and C. Whelan, "The sorcerer's apprentice guide to fault attacks," *Proc. IEEE*, vol. 94, no. 2, pp. 370–382, Feb. 2006.
- [5] S. Skorobogatov, "Optical fault masking attacks," in *Proc. Workshop Fault Diagnosis Tolerance Cryptogr.*, Aug. 2010, pp. 23–29.
- [6] J.-M. Schmidt, M. Hutter, and T. Plos, "Optical fault attacks on AES: A threat in violet," in *Proc. Workshop Fault Diagnosis Tolerance Cryptogr. (FDTC)*, 2009, pp. 13–22.
- [7] C. Aumüller, P. Bier, W. Fischer, P. Hofreiter, and J.-P. Seifert, "Fault attacks on RSA with CRT: Concrete results and practical countermeasures," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Cham, Switzerland: Springer, 2002, pp. 260–275.
- [8] S. P. Skorobogatov and R. J. Anderson, "Optical fault induction attacks," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Cham, Switzerland: Springer, 2002, pp. 2–12.
- [9] F. Zhang et al., "Persistent fault analysis on block ciphers," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2018, no. 3, pp. 150–172, Aug. 2018.
- [10] X. Li, R. Tessier, and D. Holcomb, "Precise fault injection to enable DFIA for attacking AES in remote FPGAs," in *Proc. IEEE 30th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, May 2022, pp. 1–5.
- [11] A. Caforio and S. Banik, "A study of persistent fault analysis," in *Proc. Int. Conf. Secur., Privacy, Appl. Cryptography Eng.* Cham, Switzerland: Springer, 2019, pp. 13–33.
- [12] N. Bagheri, S. Sadeghi, P. Ravi, S. Bhasin, and H. Soleimany, "SIPFA: Statistical ineffective persistent faults analysis on Feistel ciphers," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, no. 2, pp. 367–390, Jun. 2022.
- [13] C. Dobraunig, M. Eichlseder, T. Korak, S. Mangard, F. Mendel, and R. Primas, "SIFA: Exploiting ineffective fault inductions on symmetric cryptography," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2018, no. 3, pp. 547–572, Aug. 2018.
- [14] D. E. Standard, "Federal information processing standards publication 46," U.S. Dept. Commerce, Nat. Bur. Standards, Tech. Rep. FIPS PUB 46, vol. 23, 1977, pp. 1–18.
- [15] K. Aoki et al., "Camellia: A 128-bit block cipher suitable for multiple platforms—Design and analysis," in *Proc. Int. Workshop Sel. Areas Cryptogr.* Berlin, Germany: Springer, 2001, pp. 39–56.
- [16] F. Zhang, T. Feng, Z. Li, K. Ren, and X. Zhao, "Free fault leakages for deep exploitation: Algebraic persistent fault analysis on lightweight block ciphers," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, no. 2, pp. 289–311, Feb. 2022.
- [17] C. Clavier, "Secret external encodings do not prevent transient fault analysis," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Cham, Switzerland: Springer, 2007, pp. 181–194.
- [18] W. Wu and L. Zhang, "LBlock: A lightweight block cipher," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.* Cham, Switzerland: Springer, 2011, pp. 327–344.
- [19] G. Blom, L. Holst, and D. Sandell, "Problems and snapshots from the world of probability," *J. Amer. Stat. Assoc.*, vol. 89, no. 427, pp. 594–605, Jun. 1994.
- [20] H. Soleimany, N. Bagheri, H. Hadipour, P. Ravi, S. Bhasin, and S. Mansouri, "Practical multiple persistent faults analysis," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, no. 1, pp. 367–390, Nov. 2021.
- [21] N. Een, "MiniSat: A SAT solver with conflict-clause minimization," in *Proc. Int. Conf. Theory Appl. Satisfiability Test.*, 2005, pp. 203–216.
- [22] M. Soos, K. Nohl, and C. Castelluccia, "Extending SAT solvers to cryptographic problems," in *Proc. Int. Conf. Theory Appl. Satisfiability Test.* Cham, Switzerland: Springer, 2009, pp. 244–257.
- [23] L. R. Knudsen and C. V. Miolane, "Counting equations in algebraic attacks on block ciphers," *Int. J. Inf. Secur.*, vol. 9, no. 2, pp. 127–135, Apr. 2010.
- [24] X. Lou et al., "Understanding rowhammer attacks through the lens of a unified reference framework," 2019, *arXiv:1901.03538*.
- [25] M. Wang, Z. Zhang, Y. Cheng, and S. Nepal, "DRAMDig: A knowledge-assisted tool to uncover DRAM address mapping," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Kathmandu, Nepal, Jul. 2020, pp. 1–6.
- [26] M. Akkar and C. Giraud, "An implementation of DES and AES, secure against some attacks," in *Proc. 3rd Int. Workshop Cryptograph. Hardw. Embedded Syst. (CHES)*, in Lecture Notes in Computer Science, vol. 2162, Paris, France, Ç. K. Koç, D. Naccache, and C. Paar, Eds. Cham, Switzerland: Springer, May 2001, pp. 309–318, doi: 10.1007/3-540-44709-1_26.
- [27] J. Pan, F. Zhang, K. Ren, and S. Bhasin, "One fault is all it needs: Breaking higher-order masking with persistent fault analysis," in *Proc. Design, Automat. Test Eur. Conf. Exhib. Doha, Qatar: EDAA*, 2019, pp. 1–6.